Navigation, Tabs, Frames, Dialogs and File Handling

COURSE SPECIFICATIONS

Master advanced browser automation including multi-tab workflows, browser contexts, nested iframes (frameLocator), native JavaScript dialogs (alert/confirm/prompt), and resilient file upload and download pipelines.

- Track: Playwright Automation with JavaScript + AI
- Level: Intermediate to Advanced
- Focus: Multi-Page, frameLocator, Dialogs, Downloads
- Prerequisites: Days 1–6 + Surprise 1
- Architecture: Event-driven Promise.all
- Test Runner: @playwright/test (.js)

Day 7 Learning Objectives

'Master browser navigation workflows: page.goto(), history navigation (goBack/goForward), and avoiding race conditions.

'Differentiate between Browser, BrowserContext, and Page, and automate multi-tab and popup workflows reliably.

'Register context.waitForEvent("page") or page.waitForEvent("popup") listeners before triggering popup actions.

'Automate nested frames and cross-origin iframes using Playwright frameLocator() without manual frame switching.

'Handle browser dialogs (alert, confirm, prompt) using page.on("dialog") event listeners with accept() and dismiss().

'Automate single and multiple file uploads with setInputFiles() safely from test fixture directories.

'Intercept, save, and verify file downloads using the page.waitForEvent("download") event lifecycle.

Navigating between pages seems simple, but in single-page applications (SPAs) and complex multi-page apps, subtle race conditions can cause intermittent failures.

The Navigation Lifecycle

```javascript
// Basic navigation: waits until domcontentloaded by default
await page.goto('/support');

// Verifying URL change with web-first assertion:
await expect(page).toHaveURL(/.*support/);

// Verifying page title:
await expect(page).toHaveTitle(/RootAccess Support/);

// History navigation:
await page.goBack();
await page.goForward();
await page.reload();
```

Avoiding Race Conditions During Link Navigation

When clicking a link that navigates to a new page or triggers client-side routing:

- **'L Anti-Pattern: Clicking a link and immediately expecting a new page element without waiting for the URL or container to transition.**

- **' Best Practice:**

```javascript
// Click link and wait for navigation in parallel or verify URL:
await page.getByRole('link', { name: 'Knowledge Base' }).click();
await expect(page).toHaveURL(/.*knowledge-base/);
await expect(page.getByRole('heading', { name: 'Knowledge Base' })).toBeVisible();
```

Synchronized Page Navigation (Navigating to a dashboard and verifying route transition before querying inner cards.)

```
test('navigates to customer support portal', async ({ page }) => {
  await page.goto('/dashboard');
  await page.getByRole('link', { name: 'Customer Support' }).click();

  // Verify URL updated cleanly:
  await expect(page).toHaveURL('/support');
  await expect(page.getByRole('heading', { level: 1 })).toHaveText('Support Desk');
});
```

Modern web apps often open external links, PDF invoices, or OAuth authorization flows in new browser tabs (`target="_blank"`).

Understanding BrowserContext vs Page

- **Browser:** The overarching Chromium, Firefox, or WebKit process.
- **BrowserContext:** An isolated incognito-like environment with its own cookies, localStorage, and cache.
- **Page:** A single tab or window inside a BrowserContext.

Handling a New Tab Opened by Clicking a Link

To handle a newly opened tab without race conditions, Playwright requires registering the **page event listener** before clicking the link:

```javascript
test('handles link that opens a new tab', async ({ page, context }) => {
  await page.goto('/support');

  // 1. Setup the listener promise BEFORE clicking the link:
  const newPagePromise = context.waitForEvent('page');

  // 2. Click the link that has target="_blank":
  await page.getByRole('link', { name: 'Read Documentation (Opens in new tab)' }).click();

  // 3. Await the new Page object:
  const newPage = await newPagePromise;

  // 4. Wait for the new page to be ready and assert:
  await newPage.waitForLoadState();
  await expect(newPage).toHaveURL(/.*docs.rootaccess.org/);
  await expect(newPage.getByRole('heading', { level: 1 })).toHaveText('Documentation Overview');

  // 5. Interact with the new tab:
  await newPage.getByPlaceholder('Search docs').fill('locators');

  // 6. Close the new tab and return to original page:
  await newPage.close();
  await expect(page.getByRole('heading', { level: 1 })).toHaveText('Support Portal');
});
```

Popups (`page.waitForEvent('popup')`)

If an action triggers a popup window directly from a specific page:

```javascript
const popupPromise = page.waitForEvent('popup');
await page.getByRole('button', { name: 'Log in with GitHub' }).click();
const popup = await popupPromise;
await expect(popup).toHaveURL(/.*github.com/login/);
```

Opening and Validating Help Center in a New Tab (A complete test capturing a new tab, asserting its content, and returning to the main page.)

```javascript
test('opens external knowledge base in new tab', async ({ page, context }) => {
  await page.goto('/support');

  const pagePromise = context.waitForEvent('page');
  await page.getByRole('link', { name: 'Open Help Center' }).click();
  const helpPage = await pagePromise;

  await expect(helpPage).toHaveURL(/.*help-center/);
  await expect(helpPage.getByRole('heading', { name: 'Frequently Asked Questions' })).toBeVisible();

  // Cleanly close popup tab
  await helpPage.close();
});
```

An `<iframe>` (Inline Frame) embeds an independent HTML document inside another HTML page. Common use cases include Stripe credit card inputs, Google reCAPTCHA, embedded YouTube videos, and third-party customer support widgets.

Legacy Selenium vs Playwright Frames

- **Legacy Tools (Selenium):** Required explicit context switching: `driver.switchTo().frame("frameName")` followed by `driver.switchTo().defaultContent()`. If your test failed inside a frame, all subsequent tests failed because the driver was trapped inside the frame!
- **Playwright (`frameLocator`):** You **never switch contexts**. Playwright provides `frameLocator()`, which behaves like any ordinary locator. It lazy-evaluates and automatically crosses iframe boundaries seamlessly.

How to Use `frameLocator`

```javascript
// 1. Locate the iframe container using CSS, title, or name:
const paymentFrame = page.frameLocator('iframe#stripe-payment-frame');
// Or using title attribute:
const supportFrame = page.frameLocator('iframe[title="Support Chat Widget"]');

// 2. Query and interact with elements INSIDE the frame directly:
await paymentFrame.getByLabel('Card Number').fill('4242 4242 4242 4242');
await paymentFrame.getByLabel('CVC').fill('123');
await paymentFrame.getByRole('button', { name: 'Pay $49.00' }).click();

// 3. Immediately interact with the parent page without any "switch back":
await expect(page.getByRole('alert')).toHaveText('Payment successful!');
```

Nested Iframes

If an iframe contains another iframe inside it, simply chain them together:

```javascript
const innerFrame = page
  .frameLocator('iframe#outer-frame')
  .frameLocator('iframe#inner-frame');
await innerFrame.getByRole('button', { name: 'Authorize' }).click();
```

Automating an Embedded Support Form Inside an Iframe (Filling out a support inquiry inside an iframe and asserting outcome on the main page.)

```javascript
test('submits feedback inside embedded support iframe', async ({ page }) => {
  await page.goto('/support');

  // Target the embedded iframe
  const feedbackFrame = page.frameLocator('iframe[title="Customer Feedback Form"]');

  // Fill inputs inside the frame
  await feedbackFrame.getByLabel('Subject').fill('Question regarding API limits');
  await feedbackFrame.getByLabel('Message Details').fill('How do I request a rate limit increase?');
  await feedbackFrame.getByRole('button', { name: 'Send Ticket' }).click();

  // Assert confirmation inside the frame
  await expect(feedbackFrame.getByText('Ticket #1042 created')).toBeVisible();
});
```

Native JavaScript dialogs (`window.alert()`, `window.confirm()`, and `window.prompt()`) are modal popups generated by the browser engine rather than HTML DOM nodes.

Default Playwright Behavior

By default, **Playwright automatically dismisses all dialogs** so tests do not hang if an unexpected alert pops up.

If your test scenario explicitly tests dialog behavior, you must register a **dialog event listener** on the `page` object **before** triggering the action that causes the dialog:

```javascript
// 1. Handling window.alert():
page.once('dialog', async (dialog) => {
  expect(dialog.type()).toBe('alert');
  expect(dialog.message()).toBe('Your session has expired.');
  await dialog.accept(); // Dismiss the alert
});

await page.getByRole('button', { name: 'Check Session' }).click();
```

Handling Confirmations (`window.confirm`)

A confirm dialog provides OK (`accept`) and Cancel (`dismiss`):

```javascript
// Scenario A: Accept deletion (OK)
page.once('dialog', async (dialog) => {
  expect(dialog.type()).toBe('confirm');
  expect(dialog.message()).toContain('Are you sure you want to delete this repository?');
```

```
  await dialog.accept();
});
await page.getByRole('button', { name: 'Delete Project' }).click();
await expect(page.getByText('Project deleted')).toBeVisible();
// Scenario B: Cancel deletion (Cancel)
page.once('dialog', async (dialog) => {
  await dialog.dismiss();
});
await page.getByRole('button', { name: 'Delete Project' }).click();
await expect(page.getByText('Project deleted')).not.toBeVisible();
```

Handling Prompts (`window.prompt`)
A prompt accepts an input string:
```javascript
```
page.once('dialog', async (dialog) => {
  expect(dialog.type()).toBe('prompt');
  expect(dialog.defaultValue()).toBe('standard');
  await dialog.accept('premium-tier'); // Supplies text input and accepts
});
await page.getByRole('button', { name: 'Change Plan' }).click();
```

Testing Destructive Action with Confirm Dialog (Accepting a confirmation dialog before verifying item deletion.)

```
test('confirms account deactivation dialog', async ({ page }) => {
  await page.goto('/settings/security');

  page.once('dialog', async dialog => {
    expect(dialog.type()).toBe('confirm');
    expect(dialog.message()).toBe('Do you really want to deactivate your account?');
    await dialog.accept();
  });

  await page.getByRole('button', { name: 'Deactivate Account' }).click();
  await expect(page.getByRole('alert')).toHaveText('Account successfully deactivated.');
});
```

File handling is crucial in applications that manage user attachments, invoices, exports, and document parsing.

1. File Uploads with `setInputFiles`

```javascript
// Upload a single file from your project test fixtures:
await page.getByLabel('Upload Attachment').setInputFiles('tests/fixtures/sample_log.txt');

// Upload multiple files:
await page.getByLabel('Upload Documents').setInputFiles([
  'tests/fixtures/contract_part1.pdf',
  'tests/fixtures/contract_part2.pdf',
]);

// Clear/remove uploaded files:
await page.getByLabel('Upload Attachment').setInputFiles([]);
```

2. File Downloads Lifecycle

When an action triggers a file download, Playwright intercepts the download stream without opening native browser "Save As" prompts:

```javascript
test('downloads and verifies monthly billing report', async ({ page }) => {
  await page.goto('/billing');

  // 1. Setup the download event listener promise BEFORE clicking:
  const downloadPromise = page.waitForEvent('download');

  // 2. Click the download trigger:
  await page.getByRole('button', { name: 'Download CSV Export' }).click();
```

```
  // 3. Await the Download object:
  const download = await downloadPromise;

  // 4. Inspect metadata:
  expect(download.suggestedFilename()).toBe('billing_report_2026.csv');

  // 5. Save the file to a designated test output directory:
  const savePath = 'test-results/downloads/' + download.suggestedFilename();
  await download.saveAs(savePath);

  // 6. Verify file exists on disk (Node.js fs module):
  const fs = require('fs');
  expect(fs.existsSync(savePath)).toBe(true);
});
```

Complete Download Verification Test (A test triggering a CSV export, saving it to disk, and asserting its filename.)

```
const fs = require('fs');

test('exports user audit log as JSON', async ({ page }) => {
  await page.goto('/admin/audit');

  const downloadPromise = page.waitForEvent('download');
  await page.getByRole('button', { name: 'Export JSON Log' }).click();
  const download = await downloadPromise;

  expect(download.suggestedFilename()).toMatch(/audit_log_.*\.json/);

  const outputPath = 'test-results/' + download.suggestedFilename();
  await download.saveAs(outputPath);
  expect(fs.existsSync(outputPath)).toBe(true);
});
```

In this practical lab, you will combine all Day 7 concepts into a real-world multi-page customer support automation suite.

The Simulated Target Application

The application is a customer support center at `/support` with:
1. An external documentation link that opens in a new tab (`target="_blank"`).
2. An embedded ticket submission form inside an `<iframe>`.
3. A "Clear Draft" button that triggers a native `window.confirm()` dialog.
4. A file attachment input for diagnostic logs.
5. An "Export Ticket Transcript" button that triggers a `.txt` file download.

Lab Assignment Objectives (5 Test Cases):

1. **Multi-Tab: Click the documentation link, wait for the new page event, verify its URL, and close the tab.**
2. **Iframe Form: Use `page.frameLocator()` to type a support inquiry inside the embedded ticket form.**
3. **Dialog Handling: Test the "Clear Draft" button by registering a `page.once('dialog')` listener and accepting the confirmation.**
4. **File Upload: Upload a sample log file (`sample_error.log`) using `setInputFiles()`.**
5. **File Download: Trigger the transcript export, capture the download event, and verify the suggested filename with `download.suggestedFilename()`.**

Integrated Multi-Browser Interaction Snippet (A sample showing dialog handling followed by file upload.)

```
test('clears form after dialog confirmation', async ({ page }) => {
  await page.goto('/support');

  // Register dialog listener
  page.once('dialog', async dialog => {
    expect(dialog.type()).toBe('confirm');
    await dialog.accept();
  });

  await page.getByRole('button', { name: 'Clear Form' }).click();
  await expect(page.getByLabel('Ticket Title')).toHaveValue('');
});
```

dialogs, and diagnostic file upload & invoice download.

Task: Submit tests/day7-support-portal.spec.js with complete terminal logs and asynchronous event listener explanations.

Assignment Starter Code (tests/day7-support-portal.spec.js)

```
// tests/customer_support_advanced_interactions.spec.js
// DAY 7 ASSIGNMENT: Navigation, Tabs, Frames, Dialogs and File Handling
// Build an advanced test suite covering multi-tab workflows, nested iframes,
// native JavaScript dialogs, and file upload/download pipelines.

const { test, expect } = require('@playwright/test');
const path = require('path');
const fs = require('fs');

test.describe('Customer Support Portal Advanced Interactions', () => {

  test('TC-1: should open external documentation in a new tab and return to portal', async ({ page, context }) => {
    await page.goto('/support');

    // 1. Setup new page listener BEFORE clicking the link
    const newPagePromise = context.waitForEvent('page');

    // 2. Click link that opens in new tab (target="_blank")
    await page.getByRole('link', { name: 'Support Documentation' }).click();

    // 3. Await the new tab
```

10 Rubric Requirements for Day 7 Assignment:

1. Pure JavaScript structure using test and expect from @playwright/test
2. Multi-tab management using context.waitForEvent("page") or Promise.all
3. Targets iframe controls safely via page.frameLocator()
4. Sets up dialog event listener before triggering action (dialog.accept())
5. Uploads attachment files via locator.setInputFiles()
6. Validates file download via page.waitForEvent("download")
7. Executes web-first assertions across parent and secondary pages
8. Closes secondary page cleanly during teardown
9. Complete terminal test execution log showing passed status
10. Comprehensive technical explanation of multi-context events and frameLocator architecture

Q1. How does Playwright automate elements inside an iframe, and how does frameLocator() differ from Selenium switchTo().frame()?

[Difficulty: Intermediate] Evaluation Focus: Frame automation architecture and test robustness.

Suggested Answer: Playwright uses page.frameLocator("iframe_selector") to create a scoped locator for elements inside an iframe. Unlike Selenium, which mutates the global driver state via switchTo().frame() and requires a manual switchTo().defaultContent() to return, Playwright frameLocator() is purely a locator query. It auto-waits for the iframe to load and does not alter the main page context, allowing the test to seamlessly interact with both parent and iframe elements without context-switching bugs.

Follow-Up Question: "How would you locate an element inside an iframe that itself is nested inside another iframe?"

Q2. Why must context.waitForEvent("page") be declared before the action that triggers a new tab?

[Difficulty: Intermediate] Evaluation Focus: Asynchronous event listeners and race condition prevention.

Suggested Answer: In headless execution, a new tab can be created and fire its event within a few milliseconds. If the event listener is registered after the click action, the event may fire before the listener is active, causing the test to wait indefinitely and time out. By defining const pagePromise = context.waitForEvent("page") first, the listener is guaranteed to be in place when the click occurs.

Follow-Up Question: "What is the difference between context.waitForEvent("page") and page.waitForEvent("popup")?"

Q3. What is Playwright's default behavior when a native JavaScript window.alert() or window.confirm() dialog opens?

[Difficulty: Intermediate] Evaluation Focus: Dialog handling defaults and test execution resilience.

Suggested Answer: By default, Playwright automatically dismisses all dialogs (clicking Cancel or OK to dismiss) so that unexpected popups do not block the test runner. However, if a test needs to verify the dialog message or accept it, you must register a page.on("dialog") or page.once("dialog") listener before triggering the dialog, calling dialog.accept() or dialog.dismiss().

Follow-Up Question: "How do you supply text input to a window.prompt() dialog?"

Q4. Your application has a report generator that downloads a CSV file upon clicking "Export". How do you verify the file name and content in Playwright?

[Difficulty: Scenario-Based] Evaluation Focus: Download event lifecycle and filesystem verification.

Suggested Answer: First, declare const downloadPromise = page.waitForEvent("download"); before clicking the Export button. Click the button, then await downloadPromise to obtain the Download object. Verify the suggested filename with expect(download.suggestedFilename()).toBe("report.csv"). Save the file locally using await download.saveAs("test-results/report.csv"). Finally, use Node.js fs.readFileSync to inspect the CSV content and assert expected column headers and row data.

Follow-Up Question: "How does Playwright prevent downloaded files from polluting the host computer outside test directories?"

Q5. How do you upload multiple files at once in Playwright, and how do you clear an existing file selection?

[Difficulty: Intermediate] Evaluation Focus: File upload automation using setInputFiles.

Suggested Answer: To upload multiple files, pass an array of file paths to locator.setInputFiles(["file1.pdf", "file2.pdf"]). To clear an existing file input, pass an empty array: await locator.setInputFiles([]). This clears the input value and triggers the native change event.

Follow-Up Question: "What should you do if the file input is visually hidden with display:none and styled with a custom label?"

1. Which method is used in Playwright to target elements located inside an HTML <iframe>?

Key Concept: page.frameLocator() is the modern Playwright API to locate elements inside iframes without mutating global driver state.

2. What is the correct way to handle a link that opens in a new tab (target="_blank")?

Key Concept: Registering context.waitForEvent("page") before clicking guarantees that the listener catches the creation event without race conditions.

3. What is Playwright's default behavior if a native window.confirm() dialog appears and no dialog listener is registered?

Key Concept: By default, Playwright automatically dismisses dialogs to keep automated test suites running cleanly.

4. How do you accept a confirmation dialog and confirm its text in Playwright?

Key Concept: page.once("dialog", async (dialog) => { ... }) allows inspecting dialog.message() and calling dialog.accept().

5. Which method clears an already selected file from an <input type="file"> element?

Key Concept: Passing an empty array [] to setInputFiles() clears the file selection on <input type="file"> elements.

6. How do you capture and save a downloaded file to a designated test directory?

Key Concept: Waiting for the download event and calling download.saveAs(path) saves the temporary browser download to a permanent local path.

7. How do you navigate back in the browser history using Playwright?

Key Concept: await page.goBack() simulates the browser back button and waits for the resulting navigation.

8. How does Playwright handle an iframe that is nested inside another iframe?

Key Concept: Playwright allows chaining frameLocator() calls together cleanly to reach nested iframes.

9. What method returns the suggested filename that the server provided for a download?

Key Concept: download.suggestedFilename() returns the filename suggested by the HTTP Content-Disposition header or download attribute.

10. What is the relationship between BrowserContext and Page in Playwright?

Key Concept: A BrowserContext represents an isolated incognito session. Multiple Pages (tabs) can exist inside a single context, sharing cookies and storage.